



Descreva o processo de desenvolvimento de uma app móvel que funciona como uma calculadora, envolve receber números e realizar operações. Uma sugestão de resolução é perceber a que opções o utilizador tem acesso.

O utilizador pode apenas inserir números e operações.

- Como e em que ordem é que o programa deve receber estas instruções? Primeiro os números? Primeiro as operações? Pode realizar várias operações? O que significa eu clicar no ponto enquanto estou a digitar um número?



Processo de Desenvolvimento de uma Aplicação Móvel de Calculadora

Desenvolver uma aplicação móvel que funcione como uma calculadora envolve várias etapas, desde o planeamento até à implementação e testes. Abaixo, está um guia detalhado para criar essa aplicação, abordando as perguntas específicas sobre como o programa deve receber as instruções e as funcionalidades esperadas.

Etapas do Desenvolvimento

1. Planeamento e Definição de Requisitos

- **Identificar Funcionalidades Básicas:** Entrada de números, operações aritméticas básicas (adição, subtração, multiplicação, divisão), ponto decimal, limpar, e memória.
- **Definir Fluxo do Usuário:** Especificar a sequência de entradas e operações que o usuário pode realizar.

2. Design da Interface do Usuário (UI)

- **Wireframes e Mockups:** Criar esboços da interface da calculadora, incluindo a disposição dos botões numéricos e operacionais.
- **UX Considerations:** Garantir que a interface seja intuitiva e fácil de usar.

3. Escolha da Plataforma e Ferramentas de Desenvolvimento

- **Plataforma:** Decidir se a aplicação será desenvolvida para iOS, Android ou ambas.
- **Ferramentas:** Escolher a linguagem de programação e o ambiente de desenvolvimento (e.g., Swift/Xcode para iOS, Kotlin/Android Studio para Android, ou frameworks multiplataforma como Flutter).

4. Implementação do Backend Lógico

- **Classes e Métodos:** Desenvolver classes e métodos que irão gerenciar a lógica das operações aritméticas e a manipulação de entradas.

5. Desenvolvimento do Frontend (UI)

- **Botões e Visor:** Implementar os botões para números, operações, ponto decimal e outras funcionalidades, e o visor para exibir os números e resultados.

6. Integração do Frontend e Backend

- **Eventos de Botões:** Ligar os eventos dos botões aos métodos lógicos para realizar as operações.
- **Atualização do Visor:** Implementar a lógica para atualizar o visor conforme os botões são pressionados.

7. Testes e Debugging

- **Testes Unitários:** Testar individualmente os componentes e métodos para garantir que funcionem corretamente.
- **Testes de Integração:** Garantir que todos os componentes funcionem juntos como esperado.
- **Testes de UI/UX:** Realizar testes com usuários para assegurar que a interface é intuitiva e fácil de usar.

8. Lançamento e Manutenção

- **Publicação:** Submeter a aplicação para a App Store e/ou Google Play Store.
- **Manutenção:** Monitorar feedback dos usuários e lançar atualizações para corrigir bugs e adicionar melhorias.

Detalhamento das Funcionalidades e Fluxo

1. Entrada de Números

- **Primeiro Número:** O usuário começa a digitar um número; cada dígito é exibido no visor.
- **Ponto Decimal:** Ao clicar no ponto (.), ele é adicionado ao número atual, permitindo a entrada de números decimais. Apenas um ponto decimal é permitido por número.

2. Seleção de Operação

- **Operações Disponíveis:** Adição (+), Subtração (-), Multiplicação ($\times$), Divisão ($\div$).
- **Fluxo:** Após o primeiro número, o usuário escolhe a operação. A operação selecionada é armazenada, e o visor é preparado para o próximo número.

3. Entrada do Segundo Número

- O usuário digita o segundo número, que é exibido no visor.

4. Realização da Operação

- **Botão de Igual (=):** O usuário pressiona "=" para realizar a operação. O resultado é exibido no visor.

5. Ciclos de Operações

- **Novo Cálculo:** O resultado atual pode ser usado diretamente em uma nova operação. O usuário pode selecionar outra operação e repetir o processo.

6. Funcionalidades Adicionais

- **Limpar (C):** Um botão para limpar o visor e a operação atual.
- **Memória:** Funções de memória (MC, MR, M+, M-).

Exemplo de Fluxo de Uso

Exemplo 1: Operação Simples

1. Entrada do Primeiro Número:

- Usuário digita "25".
- Visor mostra "25".

2. Seleção de Operação:

- Usuário escolhe "+".
- Visor mostra "25 +".

3. Entrada do Segundo Número:

- Usuário digita "30".
- Visor mostra "25 + 30".

4. Realização da Operação:

- Usuário pressiona "=".
- Visor mostra "55".

Exemplo 2: Operação com Decimal**1. Entrada do Primeiro Número:**

- Usuário digita "7".
- Pressiona "." e depois "2".
- Visor mostra "7.2".

2. Seleção de Operação:

- Usuário escolhe "x".
- Visor mostra "7.2 x".

3. Entrada do Segundo Número:

- Usuário digita "4".
- Visor mostra "7.2 x 4".

4. Realização da Operação:

- Usuário pressiona "=".
- Visor mostra "28.8".

Pseudocódigo Simplificado

plaintext

 Copiar código

```
INICIAR CALCULADORA VISOR = "" ENQUANTO (CALCULADORA LIGADA) SE (NÚMERO
DIGITADO) ADICIONAR NÚMERO AO VISOR FIM SE SE (PONTO DECIMAL DIGITADO) SE (VISOR
NÃO CONTÉM PONTO) ADICIONAR PONTO AO VISOR FIM SE FIM SE SE (OPERAÇÃO
SELECIONADA) ARMAZENAR PRIMEIRO NÚMERO ARMAZENAR OPERAÇÃO LIMPAR VISOR FIM SE SE
(SEGUNDO NÚMERO DIGITADO) ADICIONAR NÚMERO AO VISOR FIM SE SE (IGUAL
PRESSIONADO) ARMAZENAR SEGUNDO NÚMERO CALCULAR RESULTADO EXIBIR RESULTADO NO
VISOR FIM SE SE (LIMPAR PRESSIONADO) LIMPAR VISOR E OPERAÇÕES FIM SE SE (FUNÇÃO
DE MEMÓRIA UTILIZADA) EXECUTAR FUNÇÃO DE MEMÓRIA FIM SE FIM ENQUANTO
```

Considerações Finais

Esse processo de desenvolvimento garante que a calculadora móvel seja funcional, intuitiva e fácil de usar, atendendo às necessidades básicas dos usuários ao realizar operações aritméticas. A estrutura clara e a integração das funcionalidades proporcionam uma experiência de usuário eficiente e satisfatória.